

LSPAI: An IDE Plugin for LLM-Powered Multi-Language Unit Test Generation with Language Server Protocol

Gwihwan Go

BNRist, Tsinghua University
Beijing, China
iejw1914@gmail.com

Chijin Zhou

BNRist, Tsinghua University
Beijing, China
tlock.chijin@gmail.com

Quan Zhang

BNRist, Tsinghua University
Beijing, China
quanzh98@gmail.com

Yu Jiang*

BNRist, Tsinghua University
Beijing, China
jiangyu198964@126.com

Zhao Wei

Tencent
Beijing, China
zachwei@tencent.com

Abstract

Unit testing is crucial to ensure the validity of the code, and research has been conducted to advance this domain. However, existing studies fail to address industry requirements, support for multi-language static analysis and real-time unit test generation. While integrating static analysis with a Large Language Model (LLM) could address these challenges, it typically requires effort to implement across programming languages. To address this, we propose LSPAI, an automated unit test generation tool that leverages language analysis tools and integrates them into a unified development environment via the Language Server Protocol. This approach equips LLM with multi-language static analysis capabilities, allowing a single tool to support unit test generation across multiple languages. We evaluated our method by comparing line coverage across different LLMs and programming languages, demonstrating superior performance and broad applicability. In projects, LSPAI achieved line coverage improvements of 145% for Java, 931% for Golang, and 95.62% for Python compared to Copilot. In addition, we also share our lessons learned from applying the tool in Tencent Ltd.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; Search-based software engineering.

Keywords

Unit Testing, Language Server Protocol, Large Language Model

ACM Reference Format:

Gwihwan Go, Chijin Zhou, Quan Zhang, Yu Jiang, and Zhao Wei. 2025. LSPAI: An IDE Plugin for LLM-Powered Multi-Language Unit Test Generation with Language Server Protocol. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*, June 23–28, 2025, Trondheim, Norway. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3696630.3728540>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. FSE Companion '25, June 23–28, 2025, Trondheim, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1276-0/2025/06
<https://doi.org/10.1145/3696630.3728540>

1 Introduction

Unit testing plays a pivotal role in software development by ensuring validity and reliability of code. As software systems grow in complexity, the importance of unit tests cannot be overstated, serving as a fundamental practice for identifying defects early and facilitating maintainable codebases. Extensive research has been dedicated to automating unit test generation, leading to development of Search-Based Software Testing (SBST) tools such as EvoSuite [12], Randoop [28], and Pyguint [24]. More recently, the evolution of Large Language Models (LLMs) has introduced a new paradigm for unit test generation. Models like GPT [7] and Copilot [14] can understand code context and generate relevant unit tests, significantly improving software development efficiency.

Despite significant advancements, LLMs are still prone to generating incorrect unit tests. For example, Copilot, one of the most popular tools used by many companies, is still capable of making mistakes, as acknowledged by the Copilot development team [25]. Similarly, Siddiq et al. [31] found that LLM-generated test cases show a relatively low validity rate, ranging from 2% to 12.7%, based on the SF110 [13] benchmark. As a result, researchers have proposed integrating static analysis with LLM to help them better understand the context and generate more accurate unit tests [20, 37, 39]. However, current research does not address the following two fundamental requirements of the software industry, which limits the broader adoption of these approaches in real-world settings.

First, performing static analysis across multiple programming languages is challenging. Industries adopt a variety of programming languages for different projects, and a test case generator should ideally support multiple languages. However, as shown in Table 1, most academic research has focused on one specific language rather than multi-language support. This focus stems from the difficulty of performing unified static analysis across diverse languages. Therefore, developers are forced to build customized analysis pipelines for each language, which requires significant manual adaptation effort. As a result, as far as we know, there are currently no academic tools available that can generate multi-language unit tests using static analysis.

Second, it is challenging to support the generation of real-time unit tests when integrating static analysis. Developers often write unit tests concurrently with the writing of code. However, current SBST tools and LLM-integrated tools are unsuitable for scenarios that require instant test generation, as they typically

require the compilation of entire projects to perform static analysis and collect coverage feedback. Consequently, the reliance of SBST tools on coverage feedback to enhance test case quality limits their feasibility for real-time use. This issue also persists in recent LLM integrated tools [3, 4, 20, 21, 37, 39], which depend on heavy static analysis and coverage feedback to mitigate LLM hallucinations. As a result, no academic tool currently supports real-time unit test generation, as illustrated in Table 1.

To address the aforementioned challenges, we introduce LSPAI, a real-time unit test generation tool powered by LLM and integrated with static analysis for multi-language codebases. Our key insight is that well-established language analysis tools exist for each programming language and can be accessed through the Language Server Protocol (LSP) in a unified way. By leveraging the LSP, we can perform lightweight static analysis in multiple languages within a single environment with minimal effort. Specifically, LSPAI operates in two main steps: First, LSPAI conducts dependency analysis by extracting key tokens from the focal method and retrieves the corresponding dependent source code. Second, using the retrieved dependency source code, LSPAI performs real-time unit test generation and fixing. This approach effectively leverages reliable static analysis tools to improve LLMs’ ability.

Table 1: Survey highlighting the research gap in unit test generation.

Tools	Real-Time	Multi-Language	Static Analysis Support			
			Java	Python	Golang	Others
UTGen [6], EvoSuite [12, 22, 41], Randoop [28], HITS [37], casmoda [27], testspark [30], ChatUnitTest [39]	✗	✗	✓	✗	✗	✗
PynGuin [24], CodaMosa [20], CoverUp [3], MuTAP [5], TELPA [40], SymPrompt [29], CLAP [35]	✗	✗	✗	✓	✗	✗
NxtUnitGo [36]	✗	✗	✗	✗	✓	✗
Copilot [14]	✓	✓	✗	✗	✗	✗
LSPAI	✓	✓	✓	✓	✓	✓

LSPAI brings two main benefits to developers who work in industries. First, LSPAI supports real-time unit test generation *without whole project compilation*, allowing developers to generate unit tests concurrently with code writing. Second, LSPAI simplifies the setup process by only requiring a *simple installation* of relevant language analysis plugins (e.g., extensions for Visual Studio Code), making it easily adaptable to various programming languages.

We developed LSPAI as an IDE (Integrated Development Environment) plugin for seamless integration and evaluated its performance across three widely used programming languages: Java, Python, and Golang. Our evaluation shows that LSPAI consistently improves unit test performance in terms of line coverage across real-world projects, regardless of programming language. Compared to Copilot, LSPAI achieved line coverage improvements of 145% for Java, 931% for Golang, and 95.62% for Python. When compared to a naive LLM implementation, the improvements were 122% for Java, 2,003% for Golang, and 4.84% for Python. Additionally, we share practical insights and lessons learned from applying LSPAI in an industrial setting at Tencent Ltd.

- We identified a research gap in current unit test generation: the lack of support for multi-language codebases and real-time test generation scenarios.
- We designed LSPAI as an IDE plugin, a practical tool that generates effective unit tests across multiple programming languages.

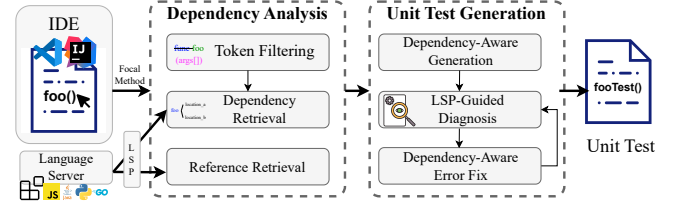


Figure 1: Overall workflow of LSPAI.

The source code is available at <https://github.com/THU-WingTecher/LSPAI> and on Zenodo [16].

- We evaluated LSPAI in real-world projects written in three programming languages, demonstrating its ability to consistently improve unit test performance. LSPAI achieved line coverage improvements of 145% for Java, 931% for Golang, and 95.62% for Python compared to Copilot.

2 Language Server Protocol

The *Language Server Protocol* (LSP) [26] is a standardized way for development tools, like text editors and IDEs, to communicate with *language servers*. A *Language Server* is meant to provide language-specific tasks, such as static analysis and code action recommendations. The main idea behind LSP is to provide a single, unified protocol that allows a language server to be used across different development environments, supporting multiple programming languages with minimal additional setup. This means that development tools can access advanced features for many programming languages through the same protocol, making it easier to work with different languages without needing to implement those features from scratch. Before LSP, features like syntax highlighting and code completion had to be written separately for each development environment and programming language. With LSP, however, editors can simply connect to a language server, which provides these features automatically, saving time and effort.

3 Design of LSPAI

This section describes the design of LSPAI, a unit test generation tool that enhances unit test creation through multi-language static analysis aided by LSP. Figure 1 illustrates LSPAI’s overall workflow. When the developer requests unit test generation for a specific method, LSPAI generates a unit test following two steps. First, LSPAI collects dependency information for the given method by extracting and retrieving token definitions. Second, it generates the unit test based on the collected dependency information. The generated unit test is then analyzed using LSP. If any issues are detected, LSPAI retrieves the necessary dependencies and corrects the errors.

3.1 Employed LSP Features

LSPAI issues standard LSP queries to apply the same static-analysis pipeline in any IDE and language. It relies on five key providers:

Symbol Provider. Returns a tree of files, classes, functions, and variables; LSPAI walks this tree to spot unit-test entry points and trace variable definitions.

Semantic Token Provider. Tags each token with its role (e.g., keyword, function call, variable read). These tags let LSPAI reason at token level and perform fine-grained analysis.

Definition Provider. Jumps from a token to its declaration. LSPAI follows these jumps to build an accurate dependency map around the target method or class.

Reference Provider. Lists every use of a symbol. LSPAI inspects these sites to understand usage patterns and make sure generated tests exercise real interactions.

Diagnosis Provider. Reports compile-time warnings and errors. LSPAI runs it on the generated tests, fixes any flagged issues, and boosts the share of valid tests.

3.2 Dependency Analysis

This module gathers dependency information to generate reliable unit tests with high coverage. The dependency information is collected in three steps: token filtering, dependency retrieval, and reference retrieval. Through this process, LSPAI acquires streamlined, high-quality dependency information.

Token Filtering. This step enhances the quality of the data to be collected by extracting tokens that are more likely to be relevant to the focal method. A focal method that requires testing is typically complex, containing numerous tokens. Analyzing every token of the method would generate a large amount of unnecessary data, most of which would not contribute to unit test generation. For example, the `parse` method in the `Parser` class of the `commons-cli` [8] project contains over 100 tokens within approximately 40 lines of code. Analyzing and retrieving information for over 100 tokens is inefficient and does not effectively enhance the quality of unit tests. Therefore, appropriate token filtering is essential. The token filtering strategy of LSPAI involves two main steps: (1) *Selecting Key Tokens*: LSPAI consider a token is important if it is given by the argument value of the method or is returned by the method. (2) *Selecting Associated Tokens*: LSPAI determine whether the tokens are associated with key tokens, utilizing the knowledge of the language server. Specifically, it requests the role of tokens that co-located with the key tokens by examining their types and modifiers through *Semantic Token Provider*. A co-located token is considered meaningful if the language server considers the token's role as declaring or defining. Following the above steps, the 100 tokens under `parse` method can be streamlined to 10 tokens. Ultimately, this module returns the extracted tokens, which LSPAI uses to retrieve further information.

Dependency Retrieval. This step involves a strategy to extract relevant dependency information from the given tokens, ensuring LSPAI retains only essential data for unit test generation while discarding unnecessary details from the language server. This is important because retrieved dependency information is often verbose, including comments, unrelated properties, or large code snippets. This can hinder unit test generation and degrade LSPAI's performance. To address this, we apply heuristic rules based on LSP knowledge. First, by requesting the *Definition Provider* using the token's position, we collect the symbol that defines the token. Next, using the *Symbol Provider*, we identify the symbol's type (e.g., function, class, method, variable, or property). Finally, we summarize the relevant code snippet based on the symbol type. For example, functions are summarized by their return type and input arguments, while methods are summarized with their return type, input arguments, and associated class member properties.

Reference Retrieval. Referring to the use case of the focal method can enhance the correctness of generated test codes. Especially for LLM, which determines its output based on context, much research [15, 38] has proved that giving an example can enhance the quality of its output. In this regard, LSPAI collects every use case of the focal method, utilizing *Reference Provider*. The collected reference information is then passed to the next step along with the dependency information and is used to generate unit test code.

3.3 Unit Test Generation

This module is responsible for generating reliable unit tests with high coverage without compiling or executing code. It leverages the given dependency information and LLM to generate unit tests. Subsequently, to mitigate the limitations of LLM, it detects issues in the generated code and fixes them.

Dependency-Aware Generation. LSPAI generates unit tests by incorporating information passed by the section 3.2. In detail, we construct the prompt incorporating the source code of the focal method, natural language description, and retrieved information. We construct our prompt template based on that of ChatUniTest [39]. Since this template is Java-specific, for generating unit tests in other programming languages, we slightly modify the prompt accordingly. The final prompt ranges from 1000 to 1,500 tokens, depending on the length of the focal method. The constructed prompt is then provided to an LLM to produce the unit test.

LSP-Guided Diagnosis. This component detects issues in the generated test code in real time without the need for compilation or execution. LLMs can produce syntactically incorrect or non-compliant code due to hallucinations. Hallucination [17, 18] refers to the generation of syntactically incorrect or semantically invalid code that deviates from the desired output. However, in a real-time generation setting, where compilation or execution is not feasible, we need alternative methods to mitigate the LLM's hallucinations. LSPAI utilizes *Diagnosis Provider* supported by LSP to inspect the generated code. If *Diagnosis Provider* does not detect any issues, LSPAI saves the unit test code; if there is any issue, LSPAI collects them and prioritizes based on severity.

Dependency-Aware Error Fix. This step integrates dependency information to fix errors effectively. Based on the diagnosis, LSPAI identifies the related symbol and retrieves necessary dependency information using the *Symbol Provider*. This information is incorporated into the prompt to assist the LLM in correcting the error alongside the necessary dependency source code. The constructed prompt is sent to the LLM to fix the code. After the fix is made, LSPAI returns to the *LSP-Guided Diagnosis* step to verify whether the issue has been resolved. If the error is fixed or the iteration limit is reached, the corrected code is saved and presented to developers.

4 Evaluation

In this section, we comprehensively evaluate LSPAI's performance on real-world projects across different programming languages.

4.1 Experiment Setup

Programming languages. We selected three different programming languages, Python, Java, and Golang, for the experiment. We selected Python and Java because their unit test generation capabilities have been extensively studied in previous research. Golang

was chosen for two reasons: (1) it is widely used in industry but has received relatively little attention in academic studies, and (2) as a relatively new language, we anticipate it presents unique challenges for LLMs in generating valid code.

Table 2: Dataset Statistics

Project	Abbr.	Domain	Version	Language
Commons-CLI [8]	CLI	Cmd-line Interface	eb541428	Java
Commons-CSV [9]	CSV	Csv file Processing	92e486ac	Java
Logrus [32]	LOG	logging for Golang	d1e6332	Golang
Cobra [11]	COB	Golang CLI interactions	3a6873e	Golang
Black [10]	BAK	Python code formatter	8dc9127	Python
Crawl4AI [34]	C4AI	LLM Friendly Web Crawler	8878b3d	Python

Baseline Selection. We selected baselines that meet the following criteria: (1) they support unit test generation across multiple programming languages, and (2) they can generate test code without compiling the entire project. Most tools listed in Table 1 do not satisfy both conditions—except for Copilot. However, the current version of Copilot supports only a limited range of LLMs [2], which restricts us to compare different models. Therefore, we implemented a baseline using the same prompt template as LSPAI, but without incorporating dependency information or LSP-guided error fixing. We refer to this version as Naive. Ultimately, we evaluated the performance of LSPAI in comparison to both Copilot and Naive.

Copilot Workflow Setting. For the Copilot implementation, we used Copilot Language Server SDK [1] and invoked the panel completion API with templated prompts. We tried our best to simulate a realistic usage scenario. Specifically, we followed developer recommendations for unit testing [2] and adopted the well-known workflow [33] of Copilot. For large-scale experiments, we automated the unit test generation process by opening the code file containing the target method, prompting Copilot to generate unit tests, and saving them using standard unit test naming conventions.

Model Selection. To demonstrate the effectiveness of LSPAI, we evaluate it using language models with different architectures and sizes. For architecture, we include both transformer-based models like the GPT series [7], and mixture-of-experts (MoE) models like DeepSeek [23] and Mistral [19]. For size variation, we test with GPT-4o, GPT-4o-M (GPT-4o-mini), and DS-V3 (Deepseek-V3).

Real-world Projects. For a fair evaluation, we selected real-world projects based on two criteria: (1) commonly used benchmarks in prior research, or (2) popular open-source projects. As shown in Table 2, we picked two projects per language. For Java, we used Commons-CLI [8] and Commons-CSV [9], both widely used in previous studies [37, 39]. For Golang, we selected Logrus [32] and Cobra [11], also referenced in related work [36]. For Python, we used Black [10], a common benchmark [20, 24], and Crawl4AI [34], a trending project with 24.6k GitHub stars. Released in May 2024, Crawl4AI is well-developed but likely unseen during LLM training, making it a strong test case for generalization. In terms of scope: Black has 472 focal methods, Crawl4AI 377, Cobra 155, Commons-CLI 140, Commons-CSV 74, and Logrus 70. The method distribution for Python is detailed in Figure 2.

4.2 Comparative Experiment

This section evaluates the performance of LSPAI using two metrics: line coverage and valid rate, where a test script is considered valid

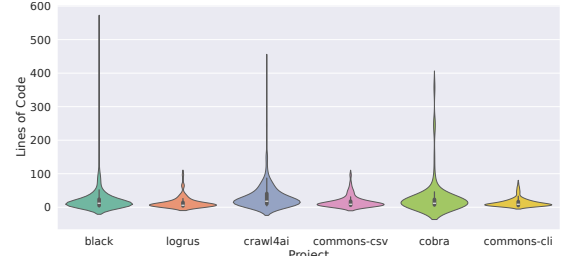


Figure 2: Focal Method Stastics for Real-World Projects.

Table 3: Comparative experimental results on line coverage and valid rate. The highest values are shown in bold.

	Model	Line Coverage			Valid Rate		
		LSPAI	NAIVE	COPILOT	LSPAI	NAIVE	COPILOT
CLI	GPT-4o	66.99%	39.92%	24.46%	77.33%	54.67%	26.00%
	GPT-4o-M	58.55%	18.78%	-	59.33%	24.00%	-
	DS-V3	61.92%	47.65%	-	79.33%	43.33%	-
CSV	GPT-4o	50.53%	35.62%	23.94%	55.71%	31.43%	14.86%
	GPT-4o-M	42.25%	17.56%	-	38.57%	6.43%	-
	DS-V3	68.26%	41.01%	-	43.57%	10.71%	-
LOG	GPT-4o	32.95%	1.16%	1.86%	21.74%	4.29%	2.86%
	GPT-4o-M	30.39%	2.78%	-	14.49%	4.29%	-
	DS-V3	54.76%	34.11%	-	40.58%	21.43%	-
COB	GPT-4o	15.75%	0.20%	5.39%	17.53%	10.32%	7.10%
	GPT-4o-M	7.52%	2.34%	-	11.04%	8.39%	-
	DS-V3	53.76%	16.36%	-	45.54%	15.65%	-
BAK	GPT-4o	50.44%	48.01%	26.95%	57.60%	47.35%	81.28%
	GPT-4o-M	38.62%	37.28%	-	51.94%	59.55%	-
	DS-V3	41.18%	40.24%	-	71.76%	67.20%	-
C4AI	GPT-4o	41.02%	39.71%	20.10%	56.52%	53.07%	84.58%
	GPT-4o-M	42.77%	38.20%	-	54.42%	66.05%	-
	DS-V3	42.84%	41.67%	-	71.35%	66.76%	-
Total		44.87%	27.02%	17.11%	50.49%	33.58%	36.11%

if it runs without any execution errors. Assertion failures are not treated as errors. As shown in Table 3, LSPAI significantly improves both line coverage and valid rate across multiple programming languages, projects, and LLMs.

Java. On average, LSPAI improves line coverage by 122% and valid rate by 149% compared to Naive, and by 145% and 262% respectively compared to Copilot. The primary reason for the increased coverage is the dependency retrieval-guided unit test generation, which effectively utilizes summarized dependency information of classes and methods to cover diverse edge cases. For the averaged valid rate, LSPAI achieves substantial improvements by 145% compared to Naive and 262% compared to Copilot. This is because of the highly structured Java program’s nature, which allows most errors to be detected before compilation through LSP. This is particularly evident in the substantial valid rate improvement observed in Java projects (e.g., CLI and CSV) compared to Naive and Copilot. These results highlight the strength of combining static analysis with LLMs in strongly typed languages like Java.

Golang. For Golang projects, LSPAI delivers the strongest improvements in unit test generation. It boosts averaged line coverage by 2,003% and valid rate by 171% over Naive, and outperforms Copilot by 931% in coverage and 403% in valid rate. LLMs like GPT often make simple mistakes in Golang tests, such as redeclaring existing objects. This causes invalid code, especially in LOG and COB projects, where Naive only achieved valid rates of 10.00% and

11.45% on average. LSPAI fixes these issues using LSP-Guided Diagnosis, raising averaged valid rates to 25.60% for LOG and 24.70% for COB. We also found that DeepSeek performs better than GPT in Golang tasks. When used with LSPAI, it excels at fixing grammar errors in test code. This may be due to DeepSeek’s pre-training with large code windows and a fill-in-the-middle objective, which helps it reuse existing code correctly. Overall, the sharp rise in both coverage and valid rate shows that LSPAI greatly improves test quality, especially for languages where LLMs tend to struggle.

Python. For Python projects like BAK and C4AI, LSPAI achieved a modest average line coverage increase of 4.84%, and increase of 1.41% in valid rate compared to NAIVE. Against Copilot, it improved averaged line coverage by 95.62% but saw a 33.17% decrease in valid rate. This trend differs from Java and Golang results in two ways. First, the coverage improvement is lower. This is because LLMs already perform well in Python—NAIVE reached the highest averaged valid rate of 59.99%. Also, Python’s dynamic nature makes it harder for LSPAI to detect errors early, limiting LSP’s effectiveness and reducing valid rates in some cases. Second, Copilot produced the highest valid rate but lowest coverage. This is because Copilot mostly generates simple assertion lines, which cause fewer errors but don’t exercise much code logic. Despite the lower valid rate, LSPAI still improved coverage by helping LLMs generate better tests using retrieved dependency information. Overall, it created tests that covered more edge cases and improved reliability.

4.3 Breakdown of LSPAI

To assess LSPAI in real-world scenarios, we measured its time and token use during unit test generation, grouped by language. Each test involved up to five fix attempts per method, with LLM access via API. We used the same project set from Table 2, and method counts are in Figure 2. All results in Table 4 were from GPT-4o.

On average, LSPAI takes 91 seconds and 4,137 tokens to generate and refine unit tests. For Golang, the results are especially strong: just 49 seconds and 6,035 tokens lead to a 20x increase in line coverage. This shows LSPAI is both efficient and effective in languages where LLMs typically struggle. For Python, however, the gains are modest. Although time and token use are reasonable (about 2 minutes and 2,000 tokens), the 4% coverage boost may not justify the cost unless test quality also improves. This makes LSPAI less suitable for Python. In summary, LSPAI performs efficiently across languages, but it shines most in areas where LLMs are weaker—like Golang—making it a powerful option for boosting test quality in challenging environments.

Table 4: Time and Token Usage by LSPAI.

	Time (milliseconds)					Token		
	Retrieval	Diagnosis	GEN	FIX	Total	GEN	FIX	Total
Java	38,578	19,168	11,669	16,194	85,789	1,541	2,866	4,407
Golang	5,925	5,337	11,177	26,938	49,377	1,150	4,885	6,035
Python	98,533	22,033	13,203	4,604	138,373	1,460	510	1,970
Averaged	47,738	15,512	12,016	15,912	91,179	1,383	2,753	4,137

5 Lessons Learned

In this section, we introduce some lessons learned during building tool for multi-language unit test generation and applying the tool to development environment in Tencent Ltd.

Alignment between Research and Industrial Needs. We identified a significant gap between academic research and industrial requirements in the domain of unit test generation. While academic efforts predominantly aim for high code coverage, they often overlook practical aspects essential for real-world usage. From our industry practice, developers urgently need a lightweight tool that can generate unit tests without whole-project compilation. Additionally, academic research has focused on specific programming languages, such as Python and Java, leaving a gap in support for other languages. For example, Golang is widely adopted in many industries, yet few academic studies have addressed unit test generation for Golang. LSPAI was developed to bridge these gaps. Although it may not achieve the same level of code coverage as academic tools like EvoSuite [12], LSPAI is designed with industrial applicability in mind, aiming for widespread use in industry scenarios.

Varying Language Proficiency of LLMs. Our experimental analysis revealed that LLMs exhibit varying levels of proficiency across programming languages, directly impacting effectiveness of LSPAI. Specifically, LLMs frequently make errors when generating Golang code, which affects the quality of generated unit tests. These findings highlight the necessity for LLM-integrated tools to adapt strategies to the specific demands and complexities of each programming language, thereby maximizing utility and effectiveness.

Demands for Better Integration Methods. This work opens several promising avenues for future research in multi-language unit test generation through the LSP. Currently, LSPAI employs prompt engineering, a low-cost but limited method for harnessing the full potential of LLMs. A more sophisticated integration method is needed to build a retrieval system that can fully exploit the capabilities of LLMs. Besides, the integration of additional language server functionalities, such as code intelligence and code action recommendations, could further enhance the accuracy and reliability of generated unit tests. In our industrial practice, we found that developers often require a more comprehensive approach to make the generated unit tests more reliable and useful.

6 Conclusion

We introduced LSPAI, a practical real-time unit test generation tool that leverages LLMs and integrates static analysis through the LSP to support multi-language codebases. LSPAI addresses the critical gap in existing research by enabling seamless unit test generation across diverse programming languages without the need for project-wide compilation, thereby facilitating concurrent test creation alongside code development. Implemented as an IDE plugin, LSPAI simplifies adoption for developers by requiring only the installation of appropriate language analysis tools. Our comprehensive evaluation of Java, Python, and Golang projects demonstrated that LSPAI consistently enhances both line coverage and valid rate compared to baseline approaches. These results highlight LSPAI’s potential to serve as a scalable and language-agnostic solution for improving test quality in modern software development.

Acknowledgements

We appreciate the reviewers’ valuable and insightful comments. This research is sponsored by CCF-Tencent Rhino-Bird Fund Program (No. 20242001274).

References

- [1] copilot-language-server-release. GitHub repository, 2025. Accessed: 2025-04-10.
- [2] Question about configuring copilot's language model. GitHub Issue, 2025. Accessed: 2025-04-10.
- [3] Juan Altmayer Pizzorno and Emery D Berger. Coverup: Coverage-guided llm-based test generation. *arXiv e-prints*, pages arXiv–2403, 2024.
- [4] Yiran Cheng, Lwin Khin Shar, Ting Zhang, Shouguo Yang, Chaopeng Dong, David Lo, Shichao Lv, Zhiqiang Shi, and Limin Sun. Llm-enhanced static analysis for precise identification of vulnerable oss versions. *arXiv preprint arXiv:2408.07321*, 2024.
- [5] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. Effective test generation using pre-trained large language models and mutation testing. *Inf. Softw. Technol.*, 171(C), July 2024.
- [6] Amirhossein Deljouyi, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. Leveraging Large Language Models for Enhancing the Understandability of Generated Unit Tests. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 392–404, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.
- [7] OpenAI et al. Gpt-4 technical report, 2024.
- [8] Apache Software Foundation. Apache commons cli, 2025. Accessed: 2025-01-15.
- [9] Apache Software Foundation. Apache commons csv, 2025. Accessed: 2025-01-15.
- [10] PSF (Python Software Foundation). Black, 2025. Accessed: 2025-01-15.
- [11] Steve Francia. Cobra. <https://github.com/spf13/cobra>, 2013. Accessed: 2025-01-15.
- [12] Gordon Fraser and Andrea Arcuri. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, pages 416–419, 2011.
- [13] Gordon Fraser and Andrea Arcuri. A large scale evaluation of automated unit test generation using evosuite. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 24(2):8, 2014.
- [14] Nat Friedman. Introducing github copilot: Your ai pair programmer, 2021. Accessed: 2024-12-20.
- [15] Tianyu Gao, Adam Fisch, and Danqi Chen. Making pre-trained language models better few-shot learners. *arXiv preprint arXiv:2012.15723*, 2020.
- [16] Gwhwan Go. Lspai. <https://zenodo.org/records/15206535>, April 2025. Presented at the FSE Industry conference in Trondheim, Norway.
- [17] Gwhwan Go, Chijin Zhou, Quan Zhang, Xiazijian Zou, Heyuan Shi, and Yu Jiang. Towards more complete constraints for deep learning library testing via complementary set guided refinement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, page 1338–1350, New York, NY, USA, 2024. Association for Computing Machinery.
- [18] Nuno M Guerreiro, Duarte Alves, Jonas Waldendorf, Barry Haddow, Alexandra Birch, Pierre Colombo, and André FT Martins. Hallucinations in large multilingual translation models. *arXiv preprint arXiv:2303.16104*, 2023.
- [19] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Th  ophile Gerv  t, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mixtral of experts, 2024.
- [20] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 919–931. IEEE, 2023.
- [21] Ziyang Li, Saikat Dutta, and Mayur Naik. Llm-assisted static analysis for detecting security vulnerabilities. *arXiv preprint arXiv:2405.17238*, 2024.
- [22] Yun Lin, Jun Sun, Gordon Fraser, Ziheng Xiu, Ting Liu, and Jin Song Dong. Recovering fitness gradients for interprocedural boolean flags in search-based testing. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 440–451, 2020.
- [23] Aixian Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Cheng-gang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [24] Stephan Lukaszczuk and Gordon Fraser. Pynguin: Automated unit test generation for python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pages 168–172, 2022.
- [25] Microsoft. Best practices for using github copilot, 2024. Accessed: 2024-12-20.
- [26] Microsoft. Language server protocol, 2024. Accessed: 2024-12-24.
- [27] Chao Ni, Xiaoya Wang, Liushan Chen, Dehai Zhao, Zhengong Cai, Shaohua Wang, and Xiaohu Yang. Casmodatest: A cascaded and model-agnostic self-directed framework for unit test generation. *arXiv preprint arXiv:2406.15743*, 2024.
- [28] Carlos Pacheco and Michael D Ernst. Randoop: feedback-directed random testing for java. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, pages 815–816, 2007.
- [29] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. *Proceedings of the ACM on Software Engineering*, 1(FSE):951–971, 2024.
- [30] Arkadii Sapozhnikov, Mitchell Olsthoorn, Annibale Panichella, Vladimir Kovalenko, and Pouria Derakhshanfar. Testspark: IntelliJ idea's ultimate test generation companion. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, pages 30–34, 2024.
- [31] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, EASE '24*, page 313–322, New York, NY, USA, 2024. Association for Computing Machinery.
- [32] Sirupsen. Logrus, 2025. Accessed: 2025-01-15.
- [33] Parth Thakkar. Copilot explorer, 2022. Accessed: 2024-12-20.
- [34] UncleCode. Crawl4ai, 2025. Accessed: 2025-01-15.
- [35] Han Wang, Han Hu, Chunyang Chen, and Burak Turhan. Chat-like asserts prediction with the support of large language model. *arXiv preprint arXiv:2407.21429*, 2024.
- [36] Siwei Wang, Xue Mao, Ziguang Cao, Yujun Gao, Qucheng Shen, and Chao Peng. Nxtunit: Automated unit test generation for go. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering, EASE '23*, page 176–179, New York, NY, USA, 2023. Association for Computing Machinery.
- [37] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. Hits: High-coverage llm-based unit test generation via method slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 1258–1268, 2024.
- [38] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [39] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. Chatunitest: a chatgpt-based automated unit test generation tool. *arXiv preprint arXiv:2305.04764*, 2023.
- [40] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. Enhancing llm-based test generation for hard-to-cover branches via static analysis. *arXiv preprint arXiv:2404.04966*, 2024.
- [41] Zhichao Zhou, Yutian Tang, Yun Lin, and Jingzhu He. An llm-based readability measurement for unit tests' context-aware inputs. *arXiv preprint arXiv:2407.21369*, 2024.

Received 2025-01-23; accepted 2025-03-25